

---

## Data Structures

---

BS (CS) \_Fall\_2025

### Lab\_02 Task



### Learning Objectives:

1. Class Templates with unit testing (Gtest)

# Class Templates in C++

## Task 0: Simple Generic Holder

Create a template class `Holder<T>` that stores a single item of any type.

### Data Members:

- **T item** : the stored value.

### Functions:

- **Constructor**: initialize the item.
- **setItem(T value)**: set the stored item.
- **getItem()**: return the stored item.
- **displayType()**: prints the data type of the stored item using `typeid(item).name()` from `typeinfo` header.

### Test Cases:

- Create `Holder<int>`, `Holder<double>`, and `Holder<string>`.
- Set and get values for each.
- Print stored type and value.

---

## Task 1:

Create a template class **MyMatrix** for performing various matrix operations with different data types e.g. `int`, `double`, `float` etc. This matrix class will handle same type of elements at a time and provide comprehensive matrix functionality. The **MyMatrix** class should consist of the following member variables:

**T matrix** A pointer to pointer (`T**`) representing a 2D dynamically allocated array to store matrix elements. **rows** An integer representing the number of rows in the matrix **cols** An integer representing the number of columns in the matrix

The **MyMatrix** class should consist of the following functions:

**Constructor** with `rows` and `columns` as parameters. This constructor should dynamically allocate memory for a 2D array and initialize all elements to zero.

**Copy Constructor** to create a deep copy of another **MyMatrix** object to avoid shallow copying issues.

**Destructor** to properly deallocate the dynamically allocated memory to prevent memory leaks.

- **inputMatrix()** which will take input from user to fill the matrix elements. This function should prompt the user to enter values row by row.
- **getElement(int row, int col)** which will return the element at specified position with proper bounds checking.
- **setElement(int row, int col, T value)** which will set the element at specified position with proper bounds checking.
- **printMatrix()** which will display the matrix in a properly formatted manner with appropriate spacing between elements.
- **isSquare()** which will return true if the matrix is a square matrix (rows == cols), otherwise false.
- **checkIdentity()** which will return true if the matrix is an identity matrix (square matrix with 1s on diagonal and 0s elsewhere), otherwise false. This function should first check if matrix is square.
- **addMatrix(const MyMatrix& other)** which will return a new MyMatrix object containing the sum of current matrix and the matrix passed as parameter. This function should check if both matrices have same dimensions before performing addition.
- **subtractMatrix(const MyMatrix& other)** which will return a new MyMatrix object containing the difference of current matrix and the matrix passed as parameter. This function should check if both matrices have same dimensions.
- **transpose()** which will return a new MyMatrix object containing the transpose of the current matrix. The transpose should have dimensions swapped (rows become cols and vice versa).

### Test Cases:

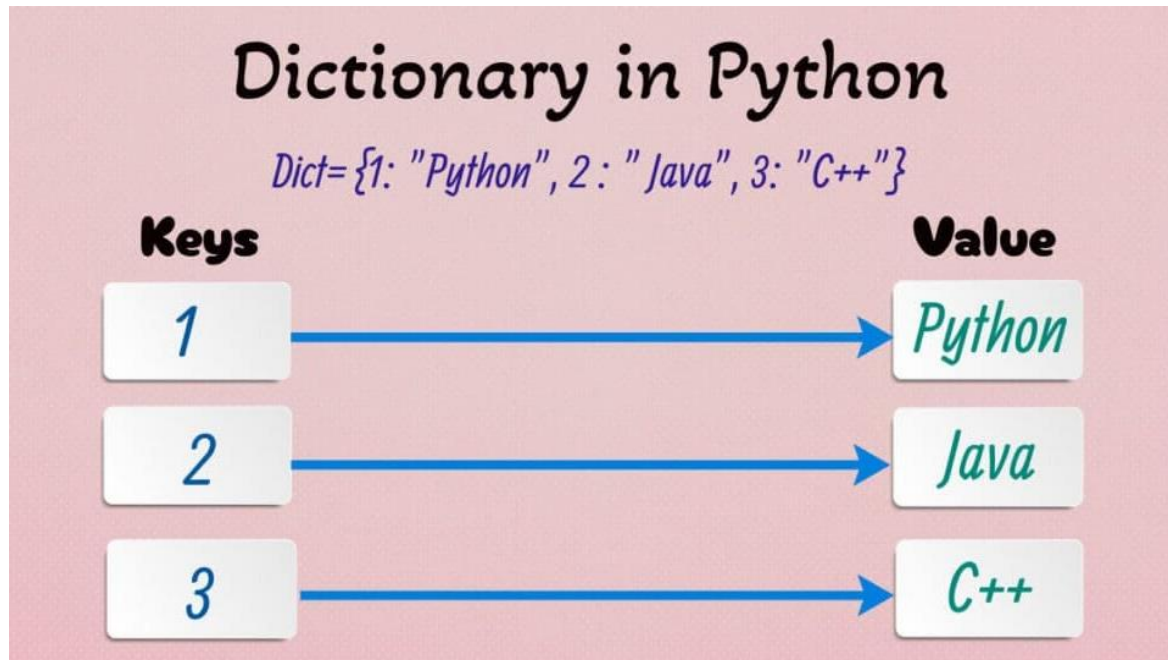
Create instances for different data types (**int, float, double**) and perform the following operations for each type:

1. Create matrices of different sizes (2x2, 3x3, 4x3)
  2. Display the matrices using printMatrix()
  3. Test identity matrix checking with both identity and non-identity matrices
  4. Test the isSquare function with square as well as non-square matrices
  5. Demonstrate matrix addition, subtraction
  6. Show transpose operation
-

## Task 2:

### Template Class Dictionary:

You are designing a generic Dictionary class similar to Python's dict that stores key–value pairs of any type.



The dictionary should be flexible enough to:

- Work with any combination of key and value types.
- Allow capacity to be set at runtime when the object is created.
- Provide default types for convenience.
- Have a special behavior when keys are std::string → make it case-insensitive by automatically converting keys to lowercase and performing searches/removals ignoring case.

The class will have Three template parameters:

- K for the key type (default: int)
- V for the value type (default: string)

### Data Members (Private)

- **K\* keys;** :- dynamic array storing keys.
- **V\* values;** :- dynamic array storing corresponding values.
- **int count;** :- current number of elements.
- **int currentSize;** :- current allocated size of arrays (starts small, doubles when needed).

## Member Functions (Public)

### Constructor

- Initializes count = 0, currentSize = 2.
- Allocates keys = new K[currentSize] and values = new V[currentSize].

### Destructor

- Deletes dynamically allocated arrays.

### isEmpty()

- Returns true if count == 0.

### insert(K key, V value)

- If the key already exists, do nothing.
- If count == currentSize, reallocate arrays with double the size.
- Insert the new key–value pair.

### search(K key)

- Returns true if key exists, otherwise false.

### remove(K key)

- If found, remove the key–value pair and shift the rest of the elements.

### getValue(K key)

- Returns the value if key exists, otherwise throws an exception.

### print()

- Prints all key–value pairs.

## Test cases :

### 1. General Dictionary Tests (Default Parameters)(K = int, V = std::string)

- Create Dictionary◇.
- Insert 3+ key value pairs:
  - Example: 1 : "Ali", 2 : "Sara", 3 : "John". Insert duplicate key (e.g., 2 : "Sara") :- should not add.
- Search existing key (2) :- returns true.
- Search non-existing key (10) :- returns false.

- Get value of existing key (2) :- returns "Sara".
- Remove key (2) :- confirm it's gone.
- Print dictionary when empty and when not empty.

## 2. **Auto resizing Tests : (K = int, V = double)**

- Create Dictionary<int, double>.
- Insert more elements than initial size (e.g., > 2) with values like 101 : 90.5, 102 : 85.3, etc.
- Verify resizing happens and all inserted elements remain accessible.

## 3. **String Key Specialization Tests (Case Insensitive)(K = std::string, V = int)**

- Create Dictionary<std::string, int>.
- Insert "Apple" : 5, "Banana" : 10.
- Search "APPLE", "apple", "ApPlE" : all return true.
- Remove "APPLE" , confirm "apple" is removed.

## 4. **Different Data Type Combination Tests**

### i) **Double keys & char values**

- Create Dictionary<double, char>.
- Insert 1.1 : 'A', 2.2 : 'B'.
- Search 2.2 , returns true.
- Remove 1.1 , confirm removal.

### ii) **Char keys & bool values**

- Create Dictionary<char, bool>.
- Insert 'x' : true, 'y' : false.
- Search 'y' , returns true.
- Get value 'x' , returns true.